`Terraform-01`

1). Install Terraform on your PC

Task 1: Terraform Installation on Windows

Date: 22 October 2026
Operating System: Windows (64-bit)
Tool: HashiCorp Terraform
Installed Version: v1.17.0
Status: ✅ Successfully Completed

---

1. Objective

To install and configure HashiCorp Terraform on a Windows machine, set up the environment variable, and verify successful installation through Git Bash.

2. What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp.

It allows DevOps engineers to create, modify, and manage infrastructure using configuration files instead of manually creating resources through cloud consoles.

Example: Instead of manually launching EC2 instances, configuring VPCs, and creating security groups in AWS, we can define these resources in Terraform configuration files.

3. Prerequisites

- Windows 64-bit operating system
- Internet connection
- Git Bash or PowerShell
- Visual Studio Code (for upcoming practical tasks)
- Permission to configure user environment variables

4. Installation Procedure

Step 1: Download Terraform

1. Open the official HashiCorp website: https://developer.hashicorp.com/terraform/install
2. Select Windows.
3. Download the AMD64 ZIP package.
4. Extract the downloaded ZIP file.

Step 2: Configure Terraform Directory

Create the following directory:

```
C:\Terraform
```

Extract the Terraform executable into this folder.

Expected file location:

```
C:\Terraform\terraform.exe
```

Step 3: Configure Environment Variables

1. Open the Windows Start menu.
2. Search for **Edit environment variables for your account**.
3. Open the Environment Variables window.
4. Under **User variables**, select the existing **Path** variable.
5. Click **Edit** → **New**.
6. Add the following directory:

```
C:\Terraform
```

7. Click OK to save the changes.
8. Close existing terminal windows and reopen Git Bash.

Purpose: Adding the Terraform directory to PATH allows the `terraform` command to run from any working directory without entering the full executable path.

Step 4: Verify Installation

Open Git Bash and execute:

terraform -version

Actual output:

Terraform v1.17.0
on windows_amd64

The output confirms that Terraform is installed and accessible through the command line.

5. Troubleshooting Encountered

Issue 1: Incorrect environment variable configuration

Initially, a new user variable named `C:\Terraform` was created, with its value also set to `C:\Terraform`.

Root cause: Creating a separate environment variable does not automatically add the Terraform executable directory to the command search path.

Resolution: Added `C:\Terraform` to the existing `Path` user variable.

Issue 2: Terraform command not found

While executing:

terraform -v

Git Bash displayed:

bash: terraform: command not found

Root cause: Git Bash was unable to locate Terraform using its available PATH configuration.

Resolution:

- Corrected the PATH configuration.
- Restarted Git Bash.
- Verified the executable using `terraform -version`.

Final result: Terraform version information displayed successfully.

6. Important Commands

Command	Purpose
`terraform -version`	Displays the installed Terraform version
`terraform -help`	Lists Terraform commands and usage information
`which terraform`	Shows which Terraform executable Git Bash resolves

`terraform init`	Initializes a Terraform working directory (next task)

Q1. What is Terraform?

Terraform is an Infrastructure as Code tool used to provision and manage infrastructure through configuration files.

Q2. Why do we configure PATH for Terraform?

PATH allows the operating system to locate the Terraform executable when a user runs a Terraform command from the terminal.

Q3. How do you verify Terraform installation?

Run `terraform -version` and confirm that the installed version is displayed.

Q4. What is the default extension for Terraform configuration files?

`.tf` is the standard extension for Terraform configuration files.

Q5. Can Terraform work with AWS, Azure, and GCP?

Yes. Terraform supports multiple cloud platforms through provider plugins.

8. Verification Evidence

Verification command:

terraform -version

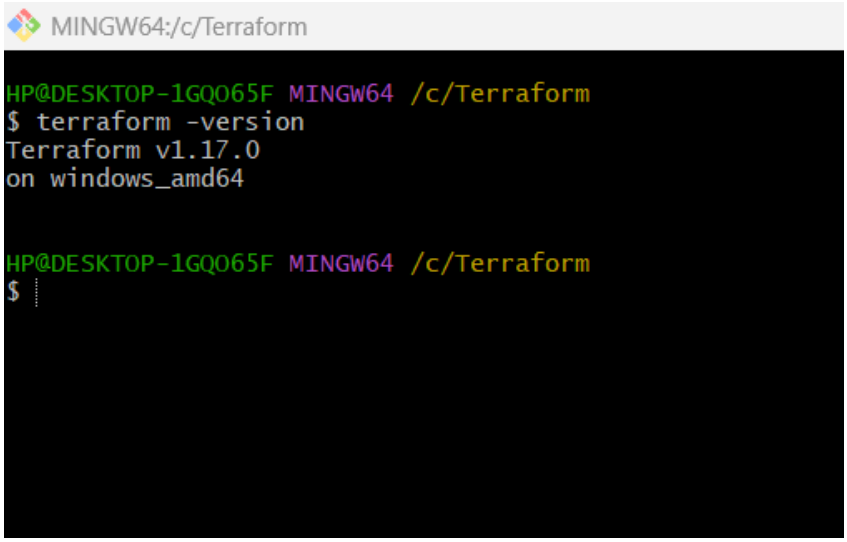
Verified environment:

Terraform v1.17.0
on windows_amd64

Evidence: Screenshot captured showing successful execution of the command in Git Bash.

9. Conclusion

Successfully installed HashiCorp Terraform v1.17.0 on Windows, configured the PATH environment variable, resolved the initial command-not-found error, and verified Terraform execution through Git Bash.



2). Execute all the templates shown in video.

1. Objective

To create, modify, inspect, and delete resources using Terraform configuration files and providers.

2. Tools Used

- Terraform CLI
- Visual Studio Code
- Windows PowerShell
- HashiCorp Local Provider v2.9.1
- HashiCorp Random Provider v3.9.1

3. Terraform Configuration

Created `main.tf` with the following configuration:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }

    random = {
      source  = "hashicorp/random"
      version = "~> 3.0"
```

```
    }
  }
}

resource "local_file" "my_pet" {
  filename = "pets.txt"
  content  = "I hate pets"
}

resource "random_pet" "my_pet" {
  prefix    = "Miss"
  separator = "."
  length    = 1
}

output "pet_name" {
  value = random_pet.my_pet.id
}
```

This is the final configuration after completing the modification exercises.

4. Practical Execution

Practical 1 — Create Local File

Executed:

terraform init
terraform plan
terraform apply

Result:

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Verified using `Get-Content .\pets.txt`.

Output: `I love pets`

Practical 2 — Modify Local File

Changed file content from `I love pets` to `I hate pets`.

Executed `terraform plan` and `terraform apply`.

Result:

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

Terraform replaced the existing local file resource.

Practical 3 — Generate Random Name

Added the Random provider and `random_pet` resource.

Executed:

terraform fmt
terraform init
terraform validate
terraform plan
terraform apply

Generated output: `Mr.guinea`

Practical 4 — Modify Random Name

Changed the prefix from `Mr` to `Miss`.

Result:

```
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

New output: `Miss.squirrel`

Practical 5 — Verify State and Idempotency

Executed:

terraform state list
terraform state show local_file.my_pet
terraform plan

Terraform tracked two managed resources.

The final plan displayed:

```
No changes. Your infrastructure matches the configuration.
```

This demonstrated idempotency.

Practical 6 — Destroy Resources

Executed:

terraform destroy

Approved the deletion using `yes`.

Result:

```
Destroy complete! Resources: 2 destroyed.
```

Verified using:

terraform state list

No resources were listed, confirming successful cleanup.

5. Error Encountered and Resolution

Error: Duplicate required providers configuration.

Reason: Added a second `required_providers` block in `main.tf`.

Resolution: Combined both Local and Random providers into one `required_providers` block.

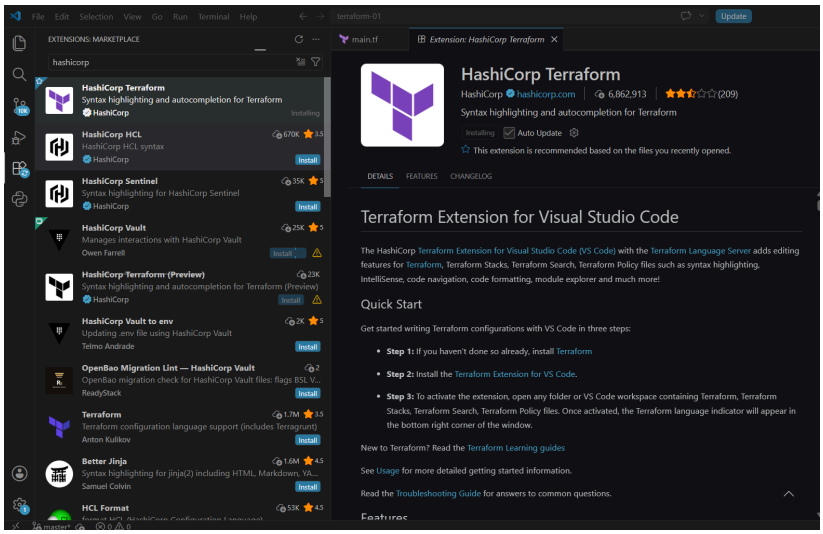
Executed `terraform init` and `terraform validate` again.

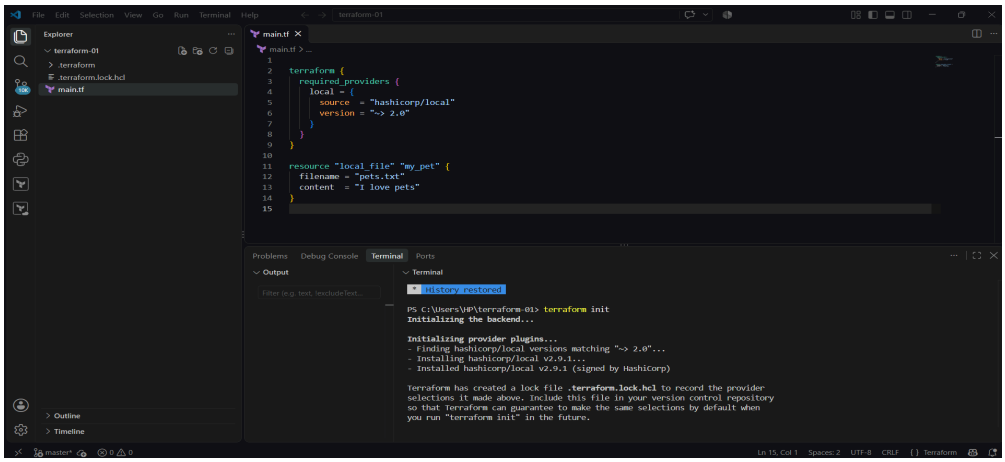
Result: Configuration successfully validated.

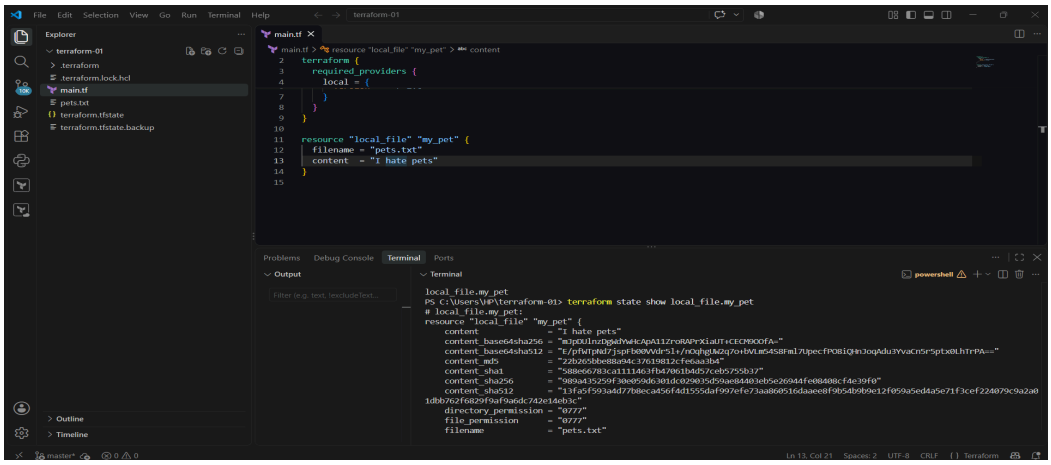
6. Conclusion

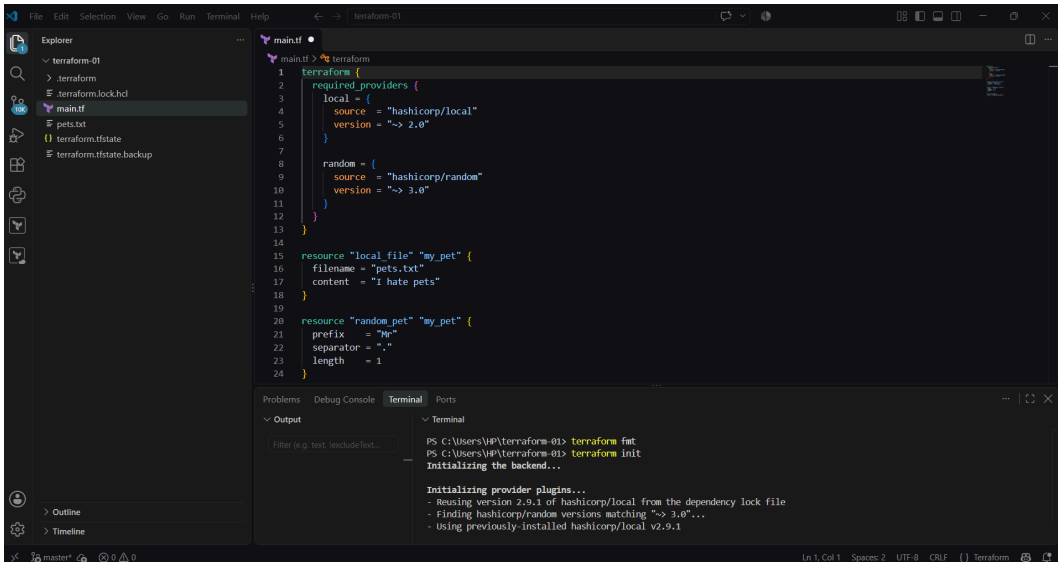
Successfully demonstrated Terraform resource creation, replacement, state management, output generation, provider installation, idempotency, and resource destruction.

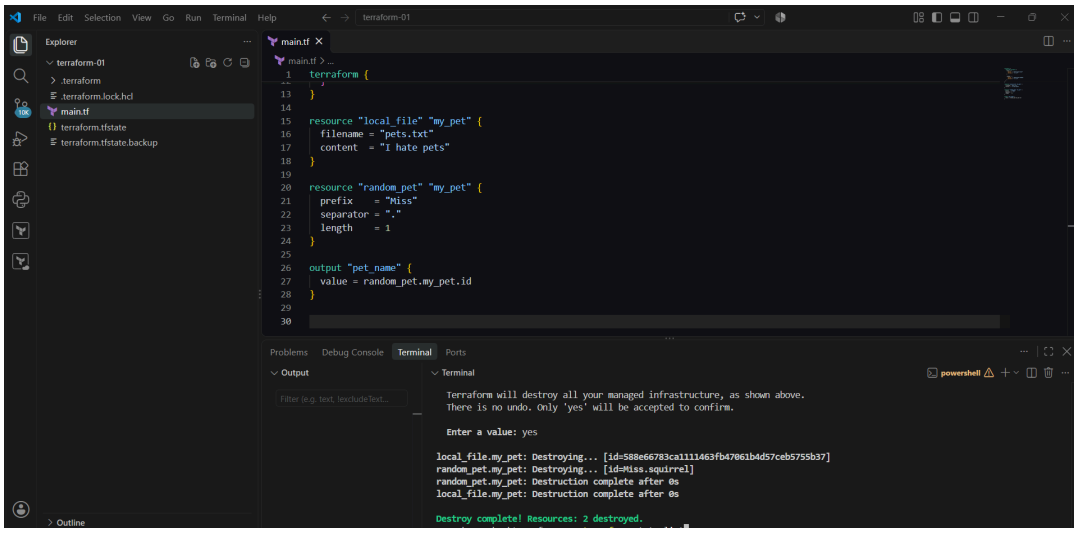
All hands-on examples from Terraform-01 training were completed and verified.


HashiCorp Terraform
Syntax highlighting and autocompletion for Terraform
HashiCorp HCL
HashiCorp Sentinel
HashiCorp Vault
HashiCorp Terraform (Preview)
HashiCorp Vault to env
OpenBao Migration Lint — HashiCorp Vault
Terraform
Better Jinja
HCL Format
HashiCorp Terraform
HashiCorp hashicorp.com | 6,862,913 (209)
Syntax highlighting and autocompletion for Terraform
Auto Update
This extension is recommended based on the files you recently opened.
DETAILS FEATURES CHANGELOG
Terraform Extension for Visual Studio Code
The HashiCorp Terraform Extension for Visual Studio Code (VS Code) with the Terraform Language Server adds editing features for Terraform, Terraform Stacks, Terraform Search, Terraform Policy files such as syntax highlighting, IntelliSense, code navigation, code formatting, module explorer and much more!
Quick Start
Get started writing Terraform configurations with VS Code in three steps:
Step 1: If you haven't done so already, install Terraform
Step 2: Install the Terraform Extension for VS Code.
Step 3: To activate the extension, open any folder or VS Code workspace containing Terraform, Terraform Stacks, Terraform Search, Terraform Policy files. Once activated, the Terraform language indicator will appear in the bottom right corner of the window.
New to Terraform? Read the Terraform Learning guides
See Usage for more detailed getting started information.
Read the Troubleshooting Guide for answers to common questions.


main.tf
terraform {
required_providers {
local = {
source = "hashicorp/local"
version = "~> 2.0"
}
}
}
resource "local_file" "my_pet" {
filename = "pets.txt"
content = "I love pets"
}
Terminal
History restored
PS C:\Users\HP\terraform-01> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.0"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.


main.tf
content = "I hate pets"
terraform {
required_providers {
local = {
}
}
}
resource "local_file" "my_pet" {
filename = "pets.txt"
content = "I hate pets"
}
Terminal
powershell
local_file.my_pet
PS C:\Users\HP\terraform-01> terraform state show local_file.my_pet
local_file.my_pet:
resource "local_file" "my_pet" {
content = "I hate pets"
content_base64sha256 = "mIpXUlnuDgaDYW4cApA11ZroRAPrXiaUT+CECM9ODfA="
content_base64sha512 = "E/pfWTpNd7jspFb00YVdr51+/nOqhgt&Zq7o+bVLm545BFml7UpecfPOBiQHnJoqAdu3YvaCn5r5ptx0LhTrPA=="
content_md5 = "22b265bbe88a94c37619812cfe6aa3b4"
content_sha1 = "588e66783ca111146d7b47061b4d57ceb5755b37"
content_sha256 = "989a435259f30e059d6301dc029035d59ae84403eb5e26944fe08408cf4e39f0"
content_sha512 = "13fa5f593a4d77b8eca456f4d1555daf997efe73aa860516daaee8f9b54b0b9e12f059a5ed4a5e71f3cef224079c9a2a01dbb762f6829f9af9a6dc742e14eb3c"
directory_permission = "0777"
file_permission = "0777"
filename = "pets.txt"





3). Note down below points, `Terraform Init` `Terraform Plan` `Terraform Apply` `Terraform Provider`.

1. Objective

To understand and document the purpose and practical usage of Terraform Init, Plan, Apply, and Provider, using the hands-on examples completed in Task 2.

2. What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp. It allows DevOps engineers to provision, modify, and manage infrastructure using configuration files written in HashiCorp Configuration Language (HCL).

Real-time example:

Instead of manually creating EC2 instances, security groups, VPCs, and load balancers through the AWS Console, DevOps engineers can define them in Terraform templates and manage them through commands.

3. Terraform Workflow

Write Terraform Code (main.tf)
|
v
terraform init
Initialize project
Download providers
|
v
terraform plan
Preview changes
|
v
terraform apply
Execute changes
|
v
Resources Created
|
v
terraform.tfstate
Track resources

4. Terraform Init

Command:

terraform init

Purpose:

The `terraform init` command initializes the Terraform working directory and prepares the project for execution.

Main functions:

- Initializes the Terraform backend.
- Downloads the required provider plugins.
- Downloads referenced modules when needed.
- Creates or updates the provider dependency lock file.
- Prepares the working directory for Terraform operations.

Practical example:

Our `main.tf` configuration required the following provider:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}
```

After executing `terraform init`, Terraform installed the Local provider.

Actual output from our task:

```
Installing hashicorp/local v2.9.1...
Installed hashicorp/local v2.9.1

Terraform has been successfully initialized!
```

Important: When we added the Random provider, we executed `terraform init` again to download its dependencies.

Interview answer:

Terraform Init initializes the working directory, configures the backend, and installs the providers and modules required by the configuration.

5. Terraform Plan

Command:

terraform plan

Purpose:

The `terraform plan` command previews the infrastructure changes Terraform intends to perform without executing those changes.

It compares the desired configuration with Terraform state and refreshed resource information, then calculates the required actions.

Practical example:

```
resource "local_file" "my_pet" {
  filename = "pets.txt"
  content  = "I love pets"
}
```

After executing `terraform plan`, we received:

Plan: 1 to add, 0 to change, 0 to destroy.

Explanation:

- `1 to add` — One new resource will be created.
- `0 to change` — No existing resources will be updated.
- `0 to destroy` — No resources will be deleted.

Terraform Plan Symbols:

Symbol	Meaning
`+`	Create resource
`-`	Destroy resource
`~`	Update resource in place
`-/+`	Destroy and recreate resource
`+/-`	Create replacement before destroying old resource

Real-time scenario:

When we changed `pets.txt` content from `"I love pets"` to `"I hate pets"`, Terraform identified that resource replacement was required.

Plan: 1 to add, 0 to change, 1 to destroy.

Interview answer:

Terraform Plan is used to preview proposed infrastructure changes before applying them. It helps identify additions, modifications, and deletions, reducing the risk of unintended changes.

6. Terraform Apply

Command:

terraform apply

Purpose:

The `terraform apply` command executes Terraform's proposed changes to create, update, replace, or delete managed resources.

Execution steps:

1. Open a terminal in the Terraform project directory.
2. Execute `terraform apply`.
3. Review the displayed execution plan.
4. Type `yes` to approve.
5. Terraform executes the operations.
6. Verify the result.

Practical output:

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s

Apply complete! Resources: 1 added,
0 changed, 0 destroyed.

Verification command:

Get-Content .\pets.txt

Output:

I love pets

Modification example:

After changing the content to `"I hate pets"`, Terraform replaced the local file.

Apply complete! Resources: 1 added,
0 changed, 1 destroyed.

Interview answer:

Terraform Apply executes the planned infrastructure changes and updates the Terraform state to track the managed resources.

7. Terraform Provider

Definition:

A Terraform Provider is a plugin that allows Terraform to interact with an external platform, service, or local system.

Providers supply resource types and data sources, and they communicate with the relevant APIs or systems.

Examples:

Provider	Purpose
`hashicorp/aws`	Manage AWS infrastructure
`hashicorp/azurerm`	Manage Microsoft Azure resources
`hashicorp/google`	Manage Google Cloud resources
`hashicorp/local`	Manage local files
`hashicorp/random`	Generate random values

Our practical example:

We used two providers in `main.tf`:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
```

```
    version = "~> 2.0"
   }

   random = {
    source  = "hashicorp/random"
    version = "~> 3.0"
   }
  }
}
```

Local Provider:

Used to create a local file called `pets.txt`.

Random Provider:

Used to generate random names using the `random_pet` resource.

```
resource "random_pet" "my_pet" {
  prefix    = "Miss"
  separator = "."
  length    = 1
}
```

Actual output from our task:

pet_name = "Miss.squirrel"

Interview answer:

A Terraform Provider is a plugin that enables Terraform to communicate with and manage resources on platforms such as AWS, Azure, GCP, or local systems.

8. Additional Terraform Commands Practiced

Command	Purpose
`terraform fmt`	Formats Terraform configuration files
`terraform validate`	Checks configuration syntax and validity
`terraform state list`	Lists resources tracked in Terraform state

`terraform state show`	Shows details of a managed resource
`terraform output`	Displays configured output values
`terraform destroy`	Destroys managed infrastructure after approval

9. Troubleshooting and Learning

Issue: Duplicate required providers configuration.

Cause: We accidentally added two `required_providers` blocks in `main.tf`.

Solution: Combined the Local and Random provider declarations into one block.

After correction:

Terraform has been successfully initialized!

Success! The configuration is valid.

Key learning: A Terraform module can contain only one `required_providers` configuration block.

10. Practical Verification Summary

Activity	Actual Result
Terraform initialization	Successful
Local provider installation	v2.9.1
Random provider installation	v3.9.1
Local file creation	`pets.txt` created
Local file modification	Successfully replaced
Random name generation	`Mr.guinea`
Random name modification	`Miss.squirrel`
State verification	Two resources tracked

Idempotency test	No changes
Resource destruction	Two resources destroyed
Final state list	Empty

11. Conclusion

Successfully learned and executed Terraform Init, Plan, and Apply commands, and understood the role of Terraform Providers.

Practiced Local and Random providers, created and modified resources, verified Terraform state, tested idempotency, and removed the resources using Terraform Destroy.

Terraform-01 Task 3 Status: COMPLETED

4). Integrate a sample Terraform template in Jenkins.

1. Objective

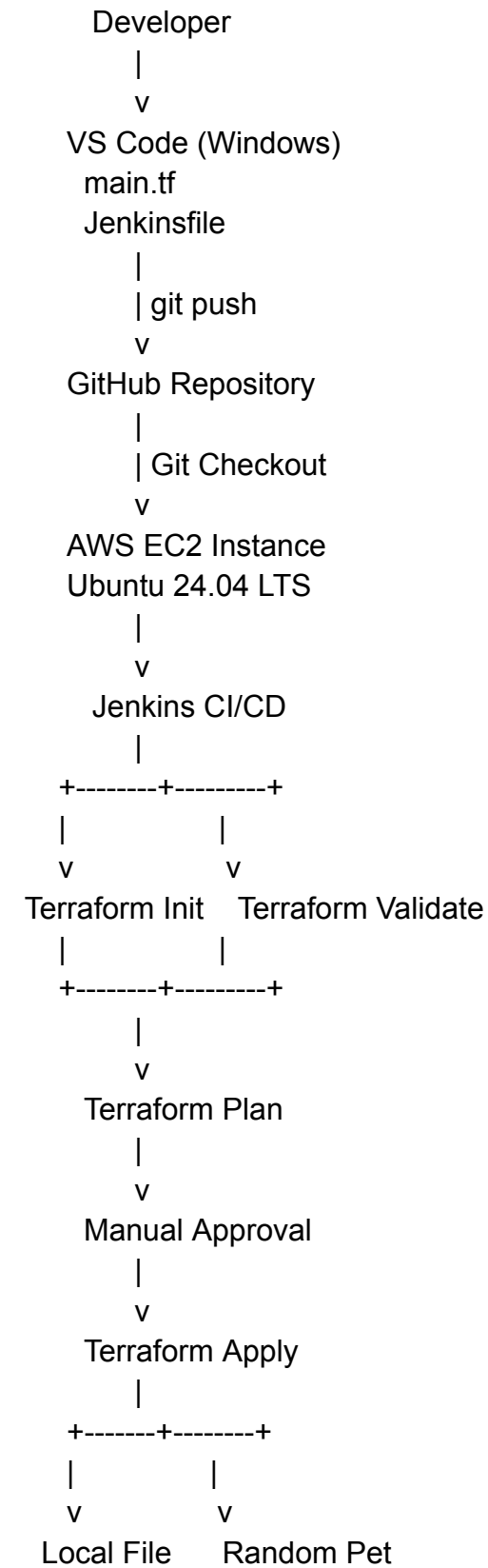
The objective of this task is to integrate Terraform with Jenkins and automate infrastructure provisioning using a Jenkins Declarative Pipeline.

The pipeline should automatically:

1. Fetch Terraform configuration files from GitHub.
2. Initialize Terraform and download required providers.
3. Validate Terraform configuration files.
4. Generate an infrastructure execution plan.
5. Request manual approval before deployment.
6. Apply the Terraform configuration.
7. Verify the resources created.

For this assignment, we used Terraform Local and Random providers to create a local file and generate a random name.

2. Architecture



pets.txt Miss.oryx
|
v
Verification

3. Tools and Technologies

Tool	Purpose
AWS EC2	Hosting Jenkins and Terraform
Ubuntu 24.04	Server operating system
Java 21	Jenkins runtime dependency
Jenkins	CI/CD pipeline automation
Terraform	Infrastructure as Code
Git	Source code version control
GitHub	Remote code repository
VS Code	Writing Terraform and Jenkinsfile configurations
Local Provider	Creating local files
Random Provider	Generating random names

---

4. Implementation Steps

Step 1: Launch an AWS EC2 Instance

1. Logged in to the AWS Management Console.
2. Opened the EC2 service.
3. Clicked **Launch Instance**.
4. Selected Ubuntu Server 24.04 LTS.
5. Configured instance networking, storage, and SSH access.
6. Allowed Jenkins access through TCP port 8080.
7. Launched the instance.

Security: SSH port 22 and Jenkins port 8080 should be accessible only from authorized IP addresses.

Connected to EC2 through EC2 Instance Connect.

Step 2: Update the Server and Install Java and Git

Executed:

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre git curl wget
```

Verified Java:

```
java -version
```

Actual output:

```
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment
```

Verified Git:

```
git --version
```

Actual output:

```
git version 2.43.0
```

Purpose: Java is required to run Jenkins, while Git is used to retrieve source code from GitHub.

Step 3: Install Jenkins

Added the Jenkins LTS repository and signing key:

```
sudo mkdir -p /etc/apt/keyrings

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
```

Configured the repository:

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list
```

Installed Jenkins:

```
sudo apt update
sudo apt install -y jenkins
```

Started Jenkins:

```
sudo systemctl enable --now jenkins
```

Checked service status:

```
sudo systemctl status jenkins --no-pager
```

Jenkins was successfully started and made accessible through its web dashboard.

Step 4: Access Jenkins Dashboard

Opened Jenkins using the EC2 public IP address and port 8080:

http://<EC2-PUBLIC-IP>:8080

Retrieved the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Completed Jenkins initial setup and accessed the dashboard.

Verification: Jenkins dashboard successfully opened, displaying Jenkins version 2.580.1.

Step 5: Install Terraform on the EC2 Server

Installed prerequisites:

```
sudo apt update
sudo apt install -y gnupg wget software-properties-common
```

Added the HashiCorp GPG signing key:

```
wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
```

Configured the HashiCorp repository:

```
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(. /etc/os-release && echo
$VERSION_CODENAME) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

Installed Terraform:

```
sudo apt update
sudo apt install -y terraform
```

Verified the installation:

```
terraform -version
```

Actual output:

```
Terraform v1.17.0
on linux_amd64
```

Verified Terraform execution under the Jenkins system user:

```
sudo -u jenkins terraform -version
```

Actual output:

```
Terraform v1.17.0
on linux_amd64
```

This confirmed that Jenkins could execute Terraform commands.

---

5. Prepare Terraform Project in VS Code

Step 6: Create Terraform Configuration

Created the project directory on the Windows PC:

```
C:\Users\HP\terraform-01
```

Created `main.tf`:

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }

    random = {
      source  = "hashicorp/random"
      version = "~> 3.0"
    }
  }
}

resource "local_file" "my_pet" {
  filename = "pets.txt"
  content  = "I hate pets"
}

resource "random_pet" "my_pet" {
  prefix    = "Miss"
  separator = "."
  length    = 1
}

output "pet_name" {
  value = random_pet.my_pet.id
}
```

Purpose:

- Local provider creates `pets.txt`.
- Random provider generates a random name.
- Output block displays the generated name.

Step 7: Create Jenkinsfile

Created a file named `Jenkinsfile` without an extension.

The Jenkins Declarative Pipeline:

```
pipeline {
    agent any

    options {
        skipDefaultCheckout(true)
        disableConcurrentBuilds()
    }

    stages {
        stage('Git Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Version') {
            steps {
                sh 'terraform -version'
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init -input=false'
            }
        }

        stage('Terraform Validate') {
            steps {
                sh 'terraform fmt -check'
                sh 'terraform validate'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan -input=false -out=tfplan'
            }
        }
```

```
        stage('Approval') {
            steps {
                input message: 'Do you want to apply Terraform changes?'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -input=false tfplan'
            }
        }

        stage('Verification') {
            steps {
                sh 'cat pets.txt'
                sh 'terraform output pet_name'
            }
        }
    }
}
```

Explanation of Pipeline Stages:

Stage	Purpose
Git Checkout	Download the code from GitHub
Terraform Version	Verify Terraform availability
Terraform Init	Initialize Terraform providers
Terraform Validate	Check formatting and configuration validity
Terraform Plan	Generate a saved execution plan
Approval	Require manual confirmation
Terraform Apply	Apply the approved plan
Verification	Check the generated file and random name

The saved `tfplan` allows Jenkins to apply the specific plan that was reviewed.

Step 8: Initialize Git and Push to GitHub

Executed on Windows:

git init
git add main.tf Jenkinsfile .gitignore .terraform.lock.hcl
git status

Committed the project:

git commit -m "Terraform-01 Jenkins pipeline integration"

Commit ID: `4af3a1e`

Renamed the branch:

git branch -M main

Connected the GitHub repository:

git remote add origin https://github.com/chandrashekar0532/terraform-01-jenkins.git

Pushed the code:

git push -u origin main

Actual Git result:

[new branch] main -> main
branch 'main' set up to track 'origin/main'.

GitHub repository successfully created and updated.

---

6. Jenkins Pipeline Configuration

Step 9: Create Pipeline Job

1. Opened Jenkins Dashboard.
2. Clicked **New Item**.
3. Selected **Pipeline**.

4. Created the job named `terraform01`.
5. Opened Pipeline configuration.

Configured the following settings:

Field	Value
Definition	Pipeline script from SCM
SCM	Git
Repository URL	https://github.com/chandrashekar0532/terraform-01-jenkins.git
Branch	`*/main`
Script Path	`Jenkinsfile`

Saved the configuration.

Step 10: Execute the Jenkins Pipeline

Clicked **Build Now**.

Opened:

Build History → Build #1 → Console Output

Jenkins successfully retrieved the Jenkinsfile and checked out Git commit `4af3a1e`.

Jenkins workspace:

/var/lib/jenkins/workspace/terraform01

The pipeline executed the following stages successfully:

- Git Checkout
- Terraform Version
- Terraform Init
- Terraform Validate
- Terraform Plan

Terraform Validate result:

Success! The configuration is valid.

Terraform Plan result:

Plan: 2 to add, 0 to change, 0 to destroy.

The execution plan included:

local_file.my_pet
random_pet.my_pet

Jenkins paused at the Manual Approval stage.

Step 11: Approve Terraform Deployment

The Jenkins approval stage displayed:

Do you want to apply Terraform changes?

Proceed or Abort

After reviewing the plan, the deployment was approved for this local-resource lab.

Terraform applied the configuration, as confirmed by the resulting state and generated file.

---

7. Final Verification

Step 12: Verify Terraform Resources on EC2

Connected to the Jenkins EC2 terminal.

Executed:

```
sudo -u jenkins bash -c '
cd /var/lib/jenkins/workspace/terraform01
terraform state list
terraform output pet_name
cat pets.txt
'
```

Actual output:

local_file.my_pet
random_pet.my_pet
"Miss.oryx"
I hate pets

Verification Results:

Verification	Actual Result	Status
GitHub checkout	Successful	PASS
Terraform v1.17.0	Available to Jenkins	PASS
Terraform initialization	Successful	PASS
Configuration validation	Successful	PASS
Terraform plan	2 resources to add	PASS
Local file	`pets.txt`	PASS
File content	`I hate pets`	PASS
Random output	`Miss.oryx`	PASS
Terraform state	2 managed resources	PASS

Result: Terraform resources were successfully provisioned and verified from the Jenkins workspace.

The complete Jenkins `Finished: SUCCESS` footer was not captured in the shared console logs, so the final job status should also be confirmed in Jenkins Build History.

---

8. Troubleshooting

Issue 1: Jenkins Git installation warning

Warning:

Selected Git installation does not exist.
Using Default

Explanation: Jenkins could not find its selected named Git installation and fell back to the default Git executable.

Result: Checkout still succeeded using Git 2.43.0.

Recommended improvement: Configure the installed Git executable in **Manage Jenkins** → **Tools**, if needed.

Issue 2: Terraform dependency lock file updated

During initialization, Terraform reported that provider dependency selections were updated.

Explanation: The dependency lock file was originally generated on Windows, while the Jenkins build ran on Linux. Platform-specific provider checksums can lead to lock file updates.

Recommended improvement: Review the lock file changes and commit any intended cross-platform checksum updates.

Issue 3: Jenkins requires Terraform on the execution machine

Terraform was already installed on the Windows PC, but Jenkins executed the pipeline on Ubuntu EC2.

Solution: Installed Terraform v1.17.0 on the EC2 server and verified access as the `jenkins` user.

Issue 4: Jenkins local state management

Terraform state for this lab is stored in the Jenkins workspace.

Limitation: If the workspace or local Terraform state is deleted, future builds may no longer have the state needed to track previously managed resources.

Production improvement: Use a secure remote Terraform backend with state locking, persistent Jenkins agents, and appropriate access controls.

Q1. Why integrate Terraform with Jenkins?

To automate infrastructure provisioning using a repeatable CI/CD pipeline, reducing manual execution and improving consistency.

Q2. What is the purpose of `checkout scm`?

It downloads the source code configured for the Jenkins pipeline from the source control system.

Q3. Why execute Terraform Init in Jenkins?

Terraform Init prepares the working directory, initializes the backend, and installs required providers and modules.

Q4. Why use Terraform Validate?

To check whether the Terraform configuration is structurally valid before planning or applying changes.

Q5. Why use `terraform plan -out=tfplan`?

It saves a specific execution plan so the Apply stage can execute the reviewed changes.

Q6. Why add a Manual Approval stage?

It allows an authorized user to review the proposed infrastructure changes before Terraform applies them.

Q7. What happens if Terraform is not installed on the Jenkins agent?

The pipeline fails when it attempts to execute the `terraform` command. Terraform must be available on the machine running that pipeline stage.

Q8. Where is Terraform state stored in this project?

In the Jenkins workspace using Terraform's local backend.

Q9. What are the limitations of using Jenkins' built-in node for deployment?

It combines Jenkins controller and build execution responsibilities. Production setups usually use dedicated agents for better isolation, reliability, and security.

Q10. How would you improve this pipeline for production?

Use dedicated Jenkins agents, a secure remote Terraform backend, least-privilege credentials, security checks, protected branches, review and approval gates, and controlled deployment environments.

---

10. Screenshots for Assignment Submission

Include these screenshots in your final document:

1. AWS EC2 instance running.
2. Java and Git version verification.
3. Jenkins dashboard.
4. Terraform version executed by Jenkins user.
5. GitHub repository containing `main.tf` and `Jenkinsfile`.
6. Jenkins Pipeline configuration with GitHub URL.
7. Jenkins Console Output showing Terraform Init and Validate.
8. Terraform Plan showing two resources to add.
9. Jenkins Manual Approval stage.
10. Jenkins build result and EC2 terminal verification displaying `Miss.oryx` and `I hate pets`.

---

11. Conclusion

Successfully integrated Terraform with Jenkins using an AWS EC2 instance running Ubuntu.

Configured Java, Git, Jenkins, and Terraform, uploaded Terraform configuration files to GitHub, and created a Jenkins Declarative Pipeline.

The pipeline checked out the source code, initialized Terraform, validated the configuration, generated an execution plan, and supported manual approval before deployment.

Verified that the Terraform-managed resources were created successfully in the Jenkins workspace.

Task 4: Terraform Deployment and Resource Verification — COMPLETED

